

Introduction

Containerized applications require a mechanism to deploy, schedule, network, scale, and maintain workloads consistently across infrastructure. **Kubernetes, Docker Swarm, and AWS Fargate address these requirements through materially different architectural models and responsibility boundaries.**

Kubernetes is a full-featured, extensible container orchestration platform. Docker Swarm provides orchestration capabilities through Docker Engine's Swarm mode, using a manager/worker architecture and Raft-based cluster state. AWS Fargate takes a different approach: it is a **serverless container compute engine** used with Amazon ECS and Amazon EKS, abstracting the underlying compute hosts from the customer.

This report compares the three technologies across **architecture, scalability, operational complexity, resource models, cost structure, and ecosystem characteristics**. The previously identified scalability, resource, storage, and pricing claims have been incorporated using the verified official documentation.

Tech Overviews

Kubernetes

Kubernetes uses a **control-plane and node architecture**. The control plane manages the cluster and includes components such as the API server, `etcd`, scheduler, and controller managers. Nodes run workload-related components including the kubelet, container runtime, and networking components.

The platform is based on **declarative desired state and continuous reconciliation**. Users specify the desired state of workloads and resources, while Kubernetes controllers continuously work toward that state.

Kubernetes is also highly extensible. Networking, storage, authentication, observability, ingress, security, and cloud integrations can be supplied through built-in functionality and external extensions.

Documented scalability

For **Kubernetes v1.37**, the official large-cluster documentation explicitly defines a design envelope consisting of all four of the following conditions:

- No more than **110 Pods per node**
- No more than **5,000 nodes**

- No more than **150,000 total Pods**
- No more than **300,000 total containers**

Kubernetes therefore explicitly supports a documented cluster design target of up to 5,000 nodes while simultaneously meeting the other stated criteria. ([Kubernetes](#))

These figures should be interpreted as documented design limits/targets, not as a guarantee of application performance. Actual capacity depends on workload characteristics, control-plane resources, networking, storage, add-ons, and cloud-provider quotas. Kubernetes' documentation also notes that large clusters may require increased cloud resource quotas and appropriately sized control-plane infrastructure. ([Kubernetes](#))

Operational model and cost

A self-managed production Kubernetes environment requires administration of control-plane infrastructure, nodes, networking, storage, security, observability, upgrades, backups, and workloads. Managed Kubernetes services can transfer some control-plane responsibilities to a provider, but workload and platform configuration remains necessary.

Kubernetes itself is open-source software and does not impose an upstream software licensing fee. Infrastructure, engineering, and optional managed-service costs remain relevant.

Kubernetes also has substantial documented adoption within the cloud-native community. The CNCF's 2025 Annual Cloud Native Survey reports that **82% of container users surveyed were running Kubernetes in production**. The figure describes the survey population rather than the entire IT industry. ([CNCF](#))

Docker Swarm

Docker Swarm, commonly referred to as Swarm mode, is integrated into **Docker Engine**. A swarm consists of Docker Engine nodes that operate as either managers or workers. ([Docker Documentation](#))

Managers perform cluster-management and orchestration functions, maintain swarm state, schedule services, and expose the Swarm API. Worker nodes execute the tasks assigned by managers. By default, managers can also execute workload tasks unless their availability is changed to **Drain**. ([Docker Documentation](#))

The principal workload abstraction is the **service**. A service defines the desired workload, including its container image, replica count, resources, networking, and placement requirements. Swarm then schedules the resulting tasks across available nodes. ([Docker Documentation](#))

Raft and manager scaling

Swarm managers use the **Raft consensus algorithm** to maintain consistent cluster state. ([Docker Documentation](#))

Docker's official documentation explicitly recommends a **maximum of seven manager nodes**. It also documents the following fault-tolerance relationships:

- **3 managers** → **maximum loss of 1 manager**
- **5 managers** → **maximum simultaneous loss of 2 managers**
- **7 managers** → **maximum loss of 3 managers**
- In general, **N managers tolerate at most $(N-1)/2$ manager failures**

Docker also explicitly warns that adding managers does **not** increase application scalability or performance; additional managers can instead increase consensus overhead. ([Docker Documentation](#))

Importantly, the seven-manager recommendation is **not a seven-node cluster limit**. Docker's administration documentation states that there is no hard limit on the number of manager nodes; the seven-node figure is Docker's recommendation, based on the trade-off between fault tolerance and Raft performance. ([Docker Documentation](#))

Workload capacity can be increased by adding worker nodes without increasing the manager quorum. ([Docker Documentation](#))

Operational model and cost

Swarm's architecture is centered on Docker Engine rather than a separate orchestration stack. A basic deployment consists of Docker Engine instances joined into a swarm, with managers handling orchestration and workers providing execution capacity.

Swarm itself does not have an official Kubernetes-equivalent published cluster target such as 5,000 nodes, 150,000 Pods, or 300,000 containers. Its documented scaling considerations instead distinguish **worker capacity** from **manager/consensus capacity**.

Infrastructure costs arise from the machines and supporting services used for the swarm, including networking, storage, registries, monitoring, and operational resources.

AWS Fargate

AWS Fargate is fundamentally different from Kubernetes and Swarm. It is **not a standalone container orchestrator**. It provides serverless compute for containers running through **Amazon ECS or Amazon EKS**. AWS describes Fargate pricing explicitly in terms of resources consumed by ECS tasks and EKS Pods. ([Amazon Web Services, Inc.](#))

The customer specifies workload resources while AWS manages the underlying compute infrastructure. This removes the need for customers to provision and maintain the underlying container hosts for Fargate workloads.

CPU and memory configurations

AWS documents specific valid Fargate CPU/memory combinations. The smallest configuration is:

- **0.25 vCPU:** 0.5 GB, 1 GB, or 2 GB — Linux

At the upper end:

- **32 vCPU:** 60 GB, 120 GB, or 244 GB — Linux, requiring Fargate Linux platform version 1.4.0 or later. ([AWS Documentation](#))

The distinction is important: saying that Fargate supports "32 vCPU with up to 244 GB" is accurate as a shorthand, but **244 GB is one of three discrete memory choices**, not an arbitrary value within a continuous 60–244 GB range. ([AWS Documentation](#))

Ephemeral storage

For Fargate Linux platform version 1.4.0 or later, each ECS task receives **20 GiB of ephemeral storage by default**, with a configurable maximum of **200 GiB**. Windows platform version 1.0.0 or later has the same 20–200 GiB range. ([AWS Documentation](#))

AWS's pricing page specifies that the default 20 GB is included and that customers pay for additional configured ephemeral storage. ([Amazon Web Services, Inc.](#))

Operational model

Fargate removes host provisioning and much of the associated host-management responsibility. Customers still configure the surrounding platform, however, including ECS or EKS, IAM, networking, security groups, load balancing, logging, service discovery, and autoscaling.

Fargate therefore changes rather than eliminates operational complexity: infrastructure-host administration is abstracted, while cloud-service and workload configuration remain customer responsibilities.

Detailed Comparison

Architectural Model

The fundamental distinction is **where orchestration occurs and where the infrastructure boundary is drawn**.

Kubernetes

Kubernetes provides the orchestration control plane itself. It maintains desired state, schedules Pods, manages services and other workload abstractions, and coordinates nodes.

The underlying compute can be customer-managed infrastructure or provider-managed infrastructure.

Docker Swarm

Swarm also provides orchestration, but its functionality is integrated into Docker Engine. Managers maintain cluster state and schedule service tasks, while worker nodes execute those tasks.

The architecture therefore remains closely tied to Docker Engine and its service/task model. ([Docker Documentation](#))

Fargate

Fargate supplies the **compute layer**, while ECS or EKS supplies the orchestration layer. Consequently, "Fargate versus Kubernetes" is not a strictly equivalent comparison: Fargate can actually be used as the compute environment for EKS Pods.

In simplified form:

- **Kubernetes:** orchestration + customer/provider-selected compute
 - **Docker Swarm:** Docker Engine-based orchestration + customer-managed compute
 - **Fargate:** AWS-managed container compute + ECS/EKS orchestration
-

Scalability

The three technologies express scalability in fundamentally different ways.

Kubernetes

Kubernetes v1.37 has an explicit large-cluster design envelope:

Metric	Documented value
Nodes per cluster	5,000 maximum
Pods per node	110 maximum
Total Pods	150,000 maximum

Total containers **300,000 maximum**

All four figures appear together in the official Kubernetes large-cluster documentation. ([Kubernetes](#))

These figures do not mean every workload will perform adequately at those levels. Kubernetes explicitly identifies control-plane sizing, cloud-provider quotas, networking, storage, and add-on resource requirements as considerations for large clusters. ([Kubernetes](#))

Docker Swarm

Swarm does not publish an equivalent four-number workload-capacity specification.

Its documented architecture separates **workload capacity** from **manager consensus capacity**. Adding worker nodes increases capacity for tasks, while manager nodes provide orchestration and fault tolerance. ([Docker Documentation](#))

Docker recommends no more than seven managers, but this should not be interpreted as a seven-node cluster maximum. Docker explicitly states that there is no hard limit on manager count and that the recommendation is a performance/fault-tolerance trade-off. ([Docker Documentation](#))

Fargate

Fargate does not require customers to construct a fleet of container hosts. Workload scaling consumes AWS-managed resources and is constrained by the valid task/Pod resource combinations and applicable AWS service quotas.

Its resource envelope is expressed at the task/Pod level rather than as a customer-managed node-cluster specification. The documented Fargate configurations range from 0.25 vCPU with 0.5/1/2 GB memory to 32 vCPU with 60/120/244 GB memory on supported Linux platforms. ([AWS Documentation](#))

Thus:

Kubernetes → explicit cluster/workload design envelope

Swarm → worker capacity plus manager consensus

Fargate → task/Pod resource allocation plus AWS quotas

None of these specifications alone establishes application performance. Workload behavior, networking, storage, scheduling, control-plane load, cloud quotas, and configuration remain important.

Setup and Operational Complexity

Kubernetes

Self-managed Kubernetes generally involves the greatest breadth of platform administration among these three models. Production environments can involve:

- Control-plane availability
- `etcd`
- Node management
- Networking
- Storage
- Identity and access control
- Security
- Observability
- Upgrades
- Backup and recovery
- Workload configuration

The complexity is closely related to Kubernetes' extensive feature set and extensibility.

Docker Swarm

Swarm has a more integrated infrastructure model because orchestration is built into Docker Engine. Administrators work with Docker Engine nodes, managers, workers, services, and tasks.

The manager quorum introduces a specific operational consideration: Raft requires a majority of managers to remain available for management operations. Existing tasks can continue running when quorum is lost, but management operations cannot proceed normally. ([Docker Documentation](#))

Fargate

Fargate removes the need to provision and maintain the underlying compute hosts. This eliminates a significant category of infrastructure administration.

It does not eliminate configuration requirements around ECS/EKS, IAM, VPC networking, security, load balancing, logging, autoscaling, and other AWS services.

The resulting distinction is:

- **Kubernetes:** substantial cluster/platform administration
 - **Swarm:** Docker-integrated cluster administration
 - **Fargate:** reduced host administration, with cloud/platform configuration remaining
-

Resource Model

The scheduling unit differs significantly.

Kubernetes schedules **Pods** onto nodes. The node layer is exposed to the cluster operator unless it is abstracted by a managed Kubernetes service.

Docker Swarm schedules **tasks** representing containers onto Docker Engine nodes. Worker nodes provide additional execution capacity. ([Docker Documentation](#))

Fargate allocates CPU and memory at the **task/Pod level**, while AWS manages the underlying hosts. AWS documents the valid resource combinations explicitly. ([AWS Documentation](#))

This creates a clear infrastructure distinction:

- Kubernetes and Swarm expose a **node infrastructure layer**.
 - Fargate largely abstracts the **host infrastructure layer**.
-

Cost Structure

Software licensing alone does not describe the total cost of operating any of the three technologies.

Kubernetes

Kubernetes is open-source software. Self-managed deployments incur costs for compute, storage, networking, operations, engineering time, monitoring, and related infrastructure. Managed Kubernetes services introduce provider-specific charges.

Docker Swarm

Swarm mode is part of Docker Engine. Costs are therefore primarily associated with the infrastructure and operational services used to run the swarm. Commercial Docker products and licensing arrangements are separate considerations from the Swarm technology itself.

Fargate

Fargate uses a consumption-based model. AWS states that customers pay for the vCPU, memory, operating system, CPU architecture, and storage resources consumed by ECS tasks or EKS Pods. Billing begins when the container image download starts and continues until task/Pod termination, subject to AWS's billing rules. ([Amazon Web Services, Inc.](#))

For **US East (N. Virginia)**, **Linux/x86**, AWS's current pricing example documents:

- **\$0.000011244 per vCPU-second**

- **\$0.000001235 per GB-memory-second**
- **\$0.0000000308 per GB-second of additional ephemeral storage** ([Amazon Web Services, Inc.](#))

The storage charge applies to storage beyond the **20 GB included by default**. AWS's example uses 30 GB total storage and therefore charges for 10 GB of additional storage. ([Amazon Web Services, Inc.](#))

These figures are **regional and configuration-specific examples**, not universal Fargate prices.

AWS also documents that Fargate pricing varies by operating system and CPU architecture, and additional charges can arise from other AWS services and data transfer. ([Amazon Web Services, Inc.](#))

Accordingly:

- **Kubernetes:** infrastructure + operational costs, with possible managed-service fees
 - **Swarm:** infrastructure + operational costs
 - **Fargate:** metered task/Pod resources + associated AWS services
-

Industry Adoption and Ecosystem

Kubernetes has a large and extensively documented cloud-native ecosystem. The 2025 CNCF Annual Cloud Native Survey reported that **82% of container users surveyed ran Kubernetes in production**. The survey also reported 98% cloud-native adoption among surveyed organizations. These are survey findings and should not be interpreted as measurements of the entire global IT market. ([CNCF](#))

Docker Swarm's ecosystem is structurally different. Its primary architectural characteristic is its close integration with Docker Engine, with services and tasks forming the central operational model. ([Docker Documentation](#))

Fargate is centered on the AWS ecosystem, particularly **Amazon ECS and Amazon EKS**. AWS explicitly describes Fargate as compute for containerized applications running on those services. ([Amazon Web Services, Inc.](#))

The three ecosystems can therefore be characterized as:

- **Kubernetes:** extensible container-orchestration ecosystem
 - **Docker Swarm:** Docker Engine-integrated orchestration
 - **Fargate:** managed AWS container-compute ecosystem
-

Final Verdict

Kubernetes, Docker Swarm, and AWS Fargate are **not simply three interchangeable container orchestrators**. They establish different boundaries between workload management, orchestration, and infrastructure management.

Kubernetes provides a comprehensive orchestration platform with an explicitly documented v1.37 design envelope of **5,000 nodes, 150,000 Pods, 300,000 containers, and 110 Pods per node**. Its architecture provides extensive control and extensibility, but self-managed deployments require substantial platform administration. ([Kubernetes](#))

Docker Swarm embeds orchestration into Docker Engine. Its manager/worker architecture uses Raft for consistent cluster state, and Docker recommends a maximum of **seven managers** while separately making clear that this is a recommendation rather than a hard cluster-size limit. Worker nodes provide additional workload capacity. ([Docker Documentation](#))

AWS Fargate shifts the infrastructure boundary toward AWS. It provides managed container compute for ECS and EKS rather than operating as a standalone orchestrator. Its resource model is task/Pod based, with documented configurations ranging from **0.25 vCPU and 0.5 GB memory to 32 vCPU with 60, 120, or 244 GB memory** on the applicable Linux platform. ([AWS Documentation](#))

Its pricing is consumption-based. The verified AWS US East (N. Virginia) Linux/x86 example uses **\$0.000011244/vCPU-second, \$0.000001235/GB-second of memory**, and **\$0.000000308/GB-second for additional ephemeral storage**, with 20 GB of ephemeral storage included by default. ([Amazon Web Services, Inc.](#))

The corrected comparison can therefore be summarized as follows:

Dimension	Kubernetes	Docker Swarm	AWS Fargate
Primary role	Container orchestration platform	Docker-integrated orchestration	Managed container compute
Orchestration	Kubernetes control plane	Docker Engine Swarm managers	ECS or EKS
Workload unit	Pod	Task/service	ECS task or EKS Pod
Infrastructure	Customer/provider managed	Customer managed	AWS managed

Documented scale fact	5,000 nodes; 150,000 Pods; 300,000 containers; 110 Pods/node	No equivalent published workload maximum; max 7 managers recommended	Task/Pod resource combinations and AWS quotas
Manager/control-plane model	Kubernetes control plane	Raft-based managers	AWS-managed infrastructure plus ECS/EKS
Maximum Fargate resource	—	—	32 vCPU; 244 GB memory option
Default Fargate ephemeral storage	—	—	20 GiB
Maximum Fargate ephemeral storage	—	—	200 GiB
Pricing model	Infrastructure/managed-service costs	Infrastructure/operational costs	Consumption-based
Key abstraction	Declarative orchestration	Docker-native orchestration	Serverless container compute

The central architectural distinction is therefore **control versus infrastructure abstraction**. Kubernetes exposes a broad orchestration model and substantial control over cluster behavior; Swarm provides Docker-integrated orchestration with manager/worker and Raft-based coordination; and Fargate abstracts the underlying container hosts while relying on ECS or EKS for orchestration.

The appropriate architecture depends on the required control boundary, workload characteristics, operational responsibilities, existing platform ecosystem, scalability requirements, and cost model. The verified facts do not establish a universal winner; they establish materially different technical and operational trade-offs.